CS1910 Computer Science I • Functions **Partner workbook**

Spotting Patterns and Generalizing

In lecture with a partner • about 25 minutes

Name: _____ **Partner:** _____

This PDF can be filled in: click in an answer space to type, or print it and write. Draw diagrams with your PDF viewer's pen or markup tool, or on paper.

What this workbook practises

Functions let you write a repeated process once and reuse it. In both problems you will look for what repeats, decide what should become a **parameter**, and decide what the function should **return**.

1 Valid hours and minutes

about 10 minutes

This code asks the user for a time. The hour must be from 1 to 12 and the minutes from 1 to 59; if the user enters an invalid value, it asks again.

```
hour = 0
while not (1 <= hour <= 12):
    hour = int(input("Enter the hour (1-12): "))

min = 0
while not (1 <= min <= 59):
    min = int(input("Enter the min (1-59): "))

print(f"Time: {hour} o'clock and {min} minutes")
```

1. What patterns do you see? Which part of the program is a repeated process?

2. What could be parameterized? What should the function return?

3. Write the function, and rewrite the program to call it twice.

2 Prime numbers between 25 and 75

about 15 minutes

From the midterm, this code decides whether 8 is prime:

```
if 8 % 2 == 0:
    print("not prime")
elif 8 % 3 == 0:
    print("not prime")
elif 8 % 4 == 0:
    print("not prime")
else:
    print("prime")
```

4. What can be generalized? Why does this code stop checking at 4?

5. The code *prints*. Why is it better for a function to *return* a value? What type should it return?

6. Write `is_prime(n)`, then use it to print all the primes from 25 to 75.

7. Which numbers does your program print?

Before you leave

about 1 minutes

- [ ] I can spot a repeated process and turn it into a function.
- [ ] I can choose parameters to generalize a function.
- [ ] I can explain why returning a value is more useful than printing it.